

Exact Inference

Hyoungh Joon, Kim

June 3, 2026

Contents

1	Introduction	2
2	Sum-product Algorithm	2
2.1	Introduction	2
2.2	Factor Graphs	2
2.3	Sum-product Algorithm	2
3	Max-sum Algorithm	5
3.1	Introduction	5
4	Junction Tree Algorithm	6
4.1	Introduction	6
5	Loopy Belief Propagation Algorithm	6
5.1	Introduction	6

1 Introduction

This document explains exact inference algorithms. Inferencing, is a process of updating the probability distribution according to the observed variables.

We'll need some knowledge about **probabilistic graphical models**, since these algorithms are based on them.

2 Sum-product Algorithm

2.1 Introduction

Sum-product algorithm can be applied to undirected tree, directed tree and polytree. The definition of **tree** and **polytree** is as following:

- **Undirected tree**: Only one path exists between nodes.
- **Directed tree**: Nodes have only one parent, and root node has no parent.
- **Directed polytree**: Nodes can have more than one parent, but still only one path (ignoring directions of the arrows) exists between nodes.

2.2 Factor Graphs

We first discuss about another graphical model, called **factor graph**.

Definition 2.2.1 (Factor Graph)

A **factor graph** is a *bipartite graph* with factor node $\{f_s\}$ and variable node $\{\mathbf{x}_k\}$ that represents following probability distribution:

$$p(\mathbf{x}) = \prod_s f_s(\mathbf{x}_s)$$

where $\mathbf{x}_s$ is a subset of variable nodes associated with a factor node f_s .

The factor graph will be helpful in understanding the sum-product algorithm.

We can reconstruct undirected tree and directed tree and polytree into factor graph, which will be a tree again. We can just add conditional probabilities in the factored distribution as factor nodes, and ordinary nodes as variable nodes.

2.3 Sum-product Algorithm

We can calculate marginal distributions by summing up useless variables. However, this is very costly if we sum up in naive way. We must exploit the conditional independency inherent in the graphical models. The **sum-product** algorithm allows us to do so. It uses **message passing**, which efficiently exploits the structure of the given probability distribution

We assume that we apply this algorithm to tree or polytree with discrete random variables. Continuous variable can use the same algorithm, changing all Σ to $\int$.

Algorithm 1: Sum-product algorithm (belief propagation)

Input : factor graph $\mathcal{G} = (\mathcal{V}, \mathcal{F}, \mathcal{E})$ of probability distribution $p(x)$ with a tree structure

Output: exact marginal distribution $p(x)$ of any variable node x

```
// initialization
1 foreach leaf node in the graph do
2   if leaf node is a variable node then
3      $\mu_{x \rightarrow f}(x) \leftarrow 1$  for each  $f \in \text{ne}(x)$ 
4   if leaf node is a factor node then
5      $\mu_{f \rightarrow x}(x) \leftarrow f(x)$  for each  $x \in \text{ne}(f)$ 

// propagation
6 while there are missing messages do
7   foreach edge  $e = (x, f) \in \mathcal{E}$  where  $x \in \mathcal{V}$  and  $f \in \mathcal{F}$  do
8     if  $x$  has received all messages then
9        $\mu_{x \rightarrow f}(x) \leftarrow \prod_{f_l \in \text{ne}(x) \setminus f} \mu_{f_l \rightarrow x}(x)$ 
10    foreach edge  $e = (x, f) \in \mathcal{E}$  where  $x \in \mathcal{V}$  and  $f \in \mathcal{F}$  do
11      if  $f$  has received all messages then
12         $\mu_{f \rightarrow x}(x) \leftarrow \left( \sum_{\text{ne}(f) \setminus x} f(x, \text{ne}(f) \setminus x) \right) \left( \prod_{x_m \in \text{ne}(f) \setminus x} \mu_{x_m \rightarrow f}(x_m) \right)$ 

// calculate marginal distributions
13 foreach node  $x \in \mathcal{V}$  do
14    $p(x) \leftarrow \prod_{f \in \text{ne}(x)} \mu_{f \rightarrow x}(x)$ 
15   normalize  $p(x)$ 
16 return  $p(x)$  about each node  $x \in \mathcal{V}$ 
```

Proof. See Bishop, 2006. The point is to exchange the product and sum. Look at equation $a(b + c) = ab + ac$. We only need two execution to calculate $a(b + c)$, while three execution is required for calculating $ab + ac$. The algorithm exploits this efficiency. $\square$

We can efficiently calculate the marginal probability of any variable x by;

1. Pass message from leaves to root;
2. Pass message from root to leaves.

We save the intermediate message passed to each node. This will use extra memory, but eliminate useless calculation, in a sense of dynamic programming.

We can also get the conditional distribution when some variables are observed:

Algorithm 2: Modified sum-product algorithm for conditional distributions

Input : factor graph $\mathcal{G} = (\mathcal{V}, \mathcal{F}, \mathcal{E})$ of probability distribution $p(x)$ with a tree structure

observed values $\{\hat{v}_1, \dots, \hat{v}_n\}$ of variable nodes $\{v_1, \dots, v_n\}$

Output: exact conditional distribution $p(x|v_1 = \hat{v}_1, v_2 = \hat{v}_2, \dots, v_n = \hat{v}_n)$ of unobserved variable node x

```
1  $p(x) \leftarrow p(x) \prod_i \delta(v_i, \hat{v}_i)$ 
2  $r \leftarrow$  sum-product algorithm about  $p(x)$ 
3 return  $r$ 
```

Multiplying by $\prod_i \delta(v_i, \hat{v}_i)$, the joint distribution equals $p(\mathbf{x} \setminus \mathbf{v}, \mathbf{v} = \hat{\mathbf{v}})$. Then, by applying sum-product algorithm, we can calculate $p(x_i | \mathbf{v} = \hat{\mathbf{v}})$ with $x_i \notin \{v_1, v_2, \dots, v_n\}$ up to a normalization constant, which can be calculated efficiently using local computation.

Calculating the marginal distribution of more than one variable is most natural when the variables are associated with same factor. If not, one has to marginalize out other variables.

1. Use the Bayes theorem: $p(x, y) = p(x|y)p(y)$;
2. Add dummy factor node f_s connecting x and y , where $f_s(x, y) = 1$.

The sum-product algorithm cannot be applied to a graph with loops, since the message passing will go to be repeated forever in the loop. However, we still can use this algorithm with loops, but it won't produce an exact result, but produce an approximate result. See section 5 for more information.

3 Max-sum Algorithm

3.1 Introduction

The **max-sum algorithm** is used to find out the MAP estimate of a given graphical model. It uses message passing, the one used for sum-product algorithm. Since multiplying small values can occur underflow, we use logarithm, which turns product into sums,

Algorithm 3: Max-sum algorithm

Input : factor graph $\mathcal{G} = (\mathcal{V}, \mathcal{F}, \mathcal{E})$ of probability distribution $p(x)$ with a tree structure
Output: the configuration x^* that maximizes the probability distribution $p(x)$

```

// initialization
1 foreach leaf node in the graph do
2   if leaf node is a variable node then
3      $\mu_{x \rightarrow f}(x) \leftarrow 0$  for each  $f \in \text{ne}(x)$ 
4   if leaf node is a factor node then
5      $\mu_{f \rightarrow x}(x) \leftarrow \log f(x)$  for each  $x \in \text{ne}(f)$ 

// propagation
6 while there are missing messages do
7   foreach edge  $e = (x, f) \in \mathcal{E}$  where  $x \in \mathcal{V}$  and  $f \in \mathcal{F}$  do
8     if  $x$  has received all messages then
9        $\mu_{x \rightarrow f}(x) \leftarrow \sum_{f_l \in \text{ne}(x) \setminus f} \mu_{f_l \rightarrow x}(x)$ 
10    foreach edge  $e = (x, f) \in \mathcal{E}$  where  $x \in \mathcal{V}$  and  $f \in \mathcal{F}$  do
11      if  $f$  has received all messages then
12         $\mu_{f \rightarrow x}(x) \leftarrow \max_{\text{ne}(f) \setminus x} [\log f(x, \text{ne}(f) \setminus x) + \sum_{x_m \in \text{ne}(f) \setminus x} \mu_{x_m \rightarrow f}(x_m)]$ 
13         $\phi_{f \rightarrow x}(x) \leftarrow \arg\max_{\text{ne}(f) \setminus x} [\log f(x, \text{ne}(f) \setminus x) + \sum_{x_m \in \text{ne}(f) \setminus x} \mu_{x_m \rightarrow f}(x_m)]$ 

14 select a root node  $x_{root} \in \mathcal{V}$ 
15  $x_{root}^* \leftarrow \arg\max_{x_{root}} [\sum_{f_l \in \text{ne}(x_{root})} \mu_{f_l \rightarrow x_{root}}(x_{root})]$ 
16 while moving down from root to leaf nodes do
17   foreach factor  $f \in \text{ne}(x)$  where  $x$  knows its configuration  $x^*$  do
18      $X_{\text{ne}(f) \setminus x}^* \leftarrow \phi_{f \rightarrow x}(x^*)$ 

19 return  $x^*$  for all variable node  $x \in \mathcal{V}$ 

```

We save each values that maximizes the whole probability distribution, to reduce the useless repeated operations when obtaining $\mathbf{x}^{\max}$.

Remark 3.1.1 (Message Passing is a Framework)

As one can see, by changing the content of the message and the operations, we can yield different algorithms. Sum-product algorithm and max-sum algorithms are the examples. Dijkstra algorithm and max flow algorithm can be interpreted as message passing framework. One would think of the factor graph's absence, but factor graph is only a tool for explaining the message passing framework. For more information, one may search for algebraic path problem and semiring.

4 Junction Tree Algorithm

4.1 Introduction

The **junction-tree algorithm** is an algorithm that calculates marginal distributions like sum-product algorithm. The difference is that it can be applied to any type of graphs, in the expense of computational memories. TODO: Junction Tree Algorithm

5 Loopy Belief Propagation Algorithm

5.1 Introduction

TODO: Loopy Belief Propagation Algorithm *To be continued....*